

1.1. Architecture hardware à prendre en considérationⁱ

Sur la carte fournie, le pld altera est accessible depuis le uC mbed via un port // de donnée bidirectionnel 8 bits (D7..D0) , un signal de direction de l’échange (RD_WRn) qui vaut 1 en lecture et 0 en écriture, et un signal de validation CSn qui vaut 0 lorsque la donnée peut être échangée.

Une classe de communication (Bloc_IO.cpp et Bloc_IO.h) a donc été créée pour vous permettre de réaliser cette communication

Cette classe est téléchargeable ici : https://developer.mbed.org/users/superphil06/code/bloc_io/#7b4e6771a530

Cette class possède un constructeur : Bloc_IO dans lequel on précise dans l’ordre

- La broche utilisée pour jouer le rôle de RD_WRn,
- La broche utilisée pour jouer le rôle de CSn.
- Les 8 broches du LPC1768 utilisées pour communiquer la donnée (lsb en tête).

1.2. Utilisation de la classe Bloc_IO :

- a) Il faut d’abord instancier l’objet qui nous servira à communiquer

Exemple de déclaration et instantiation d’un objet de cette classe:

```
Bloc_IO MyPLD(p25,p26,p5,p6,p7,p8,p9,p10,p23,p24) ;
```

Ici on vient de créer l’objet MyPLD.

Broche RD_WRn :p25

Broche CSn :p26

Bus de données : p5(LSB),p6,p7,p8,p9,p10,p23,p24 (MSB)

- b) Comment écrire une donnée 8 bits dans le PLD ?

Il faut utiliser la méthode .write appliquée à notre objet et lui passer la valeur à écrire en paramètre

Exemple d’écriture de la valeur 0x55 dans le pld :

```
MyPLD.write(0x55); // write 0x55 onto pld
```

- c) Comment lire une donnée 8 bits en provenance du PLD ?

Il faut utiliser la méthode .read qui renvoie une valeur 8 bits non signée.

Exemple de lecture du PLD et affectation d’une variable locale byVal. :

```
byVal=MyPLD.read(); // read from pld
```